

# EXAM THEORY NOTES — UNIT 1 & UNIT 2

**Course Pattern:** TE IT - 2024 Pattern (SPPU)  
**Source Material:** Prof. Sonal Kulkarni (IT) Slides

**Academic Term:** Semester V (In-Sem Exam)  
**Course Code:** PCC-303-ITT

## UNIT 1: FINITE AUTOMATA

### 1.1 Deterministic Finite Automata (DFA)

- **Core Characteristics:**

- In a DFA, there is **only one path/transition** for a given input from the current state to the next state.
- A DFA **does not accept null moves** (it cannot change state without reading an input symbol).
- A DFA can contain **multiple final states**.

- **Formal 5-Tuple Definition of DFA:** A Deterministic Finite Automaton (DFA) is defined as a collection of 5 tuples:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- $Q$ : Finite set called **states**.
- $\Sigma$ : Finite set called **alphabets** (input symbols).
- $\delta : Q \times \Sigma \rightarrow Q$ : **Transition function** (maps state and input symbol to a unique next state).
- $q_0 \in Q$ : **Start or initial state**.
- $F \subseteq Q$ : Set of **final or accept states**.
- **Graphical Representation of DFA (State Diagram Digraphs):**
  - **Vertices:** Represent internal states.
  - **Arcs (Directed Edges):** Labeled with an input character to show state transitions.
  - **Start State:** Represented with an incoming arrow pointing from nowhere.
  - **Final State:** Represented by a **double circle**.
- **Trap State / Dead State:**
  - If a transition goes to a state from which the automaton can **never escape** and from which it **cannot reach any final state**, such a state is called a **trap state** or **dead state**.

### 1.2 Non-Deterministic Finite Automata (NFA) & Epsilon ( $\epsilon$ ) Transitions

- **Core Characteristics of NFA:**

- Finite automata are called NFA when there exist **many paths** for a specific input from the current state to the next state.
- **Every NFA is not a DFA, but each NFA can be translated into an equivalent DFA.**
- NFA differs from DFA by two key exceptions:
  1. It contains **multiple next states** for a single input symbol.
  2. It can contain **null ( $\epsilon$ ) transitions**.

- **Formal 5-Tuple Definition of NFA (NFA):**

$$M = (Q, \Sigma, \delta, q_0, F)$$

- $Q$ : Finite set of states.
- $\Sigma$ : Finite set of input symbols (alphabets).
- $\delta : Q \times \Sigma \rightarrow 2^Q$ : Transition function mapping to the **power set of  $Q$  ( $2^Q$ )**, because transitions can occur to any combination/subset of  $Q$  states.

- $q_0 \in Q$ : Initial start state.
- $F \subseteq Q$ : Set of final states.
- **Epsilon ( $\epsilon$ ) Transition Function:**
  - $\epsilon$ -transitions make a machine non-deterministic.
  - When a machine enters a state with an  $\epsilon$ -transition exiting it, the machine can choose to either stay in that state and await the next input symbol, or **immediately take the  $\epsilon$ -transition to enter a new state without consuming any input symbol.**

### 1.3 Difference Between DFA and NFA

| Parameter              | DFA (Deterministic)                                                                         | NFA (Non-Deterministic)                                                            |
|------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Epsilon Moves</b>   | DFA cannot use Epsilon ( $\epsilon$ ) transitions.                                          | NFA can use Epsilon ( $\epsilon$ ) transitions.                                    |
| <b>Machine Model</b>   | DFA can be understood as one machine.                                                       | NFA can be understood as multiple state machines computing at the same time.       |
| <b>Next States</b>     | For each state and input symbol, there is exactly one possible next state.                  | For each state and input symbol, can have zero, one, or more possible next states. |
| <b>Construction</b>    | DFA is more difficult to construct.                                                         | NFA is easier to construct.                                                        |
| <b>Rejection Rule</b>  | DFA rejects the string in case it terminates in a state different from the accepting state. | NFA rejects the string in the event of all branches dying or refusing the string.  |
| <b>Execution Time</b>  | Time needed for executing an input string is <b>less</b> (faster).                          | Time needed for executing an input string is <b>more</b> .                         |
| <b>Subset Relation</b> | All DFA are NFA.                                                                            | Not all NFA are DFA.                                                               |
| <b>Space Needed</b>    | DFA requires <b>more space</b> (more states).                                               | NFA requires <b>less space</b> than DFA.                                           |

### 1.4 Conversion from NFA to DFA (Subset Construction Steps)

#### STEPS:

##### Step-by-Step Procedure:

- **Step 1:** Initially set  $Q' = \emptyset$ .
- **Step 2:** Add start state  $q_0$  of NFA as  $[q_0]$  to  $Q'$ . Then find the transitions from this start state for all input symbols.
- **Step 3:** In  $Q'$ , find the possible set of states for each input symbol. If this generated set of states is not already present in  $Q'$ , add it to  $Q'$ .
- **Step 4:** Repeat Step 3 for all newly added states until no new states are generated in the transition table.
- **Step 5:** In the constructed DFA, the final states will be **all composite states that contain at least one final state of the NFA ( $F$ )**.

### 1.5 Finite Automata with Output: Moore and Mealy Machines

Moore and Mealy Machines are **Transducers** that produce outputs based on the input of the current state or previous state.

- **1. Moore Machine:**
  - A finite state machine where the **output value depends only on the present state.**
  - Defined as a 6-tuple:  $M = (Q, q_0, \Sigma, O, \delta, \lambda)$ 
    - $Q$ : Finite set of states.
    - $q_0$ : Initial state.
    - $\Sigma$ : Input alphabet.

- $O$ : Output alphabet.
- $\delta : Q \times \Sigma \rightarrow Q$ : Transition function.
- $\lambda : Q \rightarrow O$ : Output function mapping **state to output**.

## • 2. Mealy Machine:

- A finite state machine where the **output value depends on both the present state and current input symbol**.
- Defined as a 6-tuple:  $M = (Q, q_0, \Sigma, O, \delta, \lambda')$ 
  - $Q, q_0, \Sigma, O, \delta$ : Same as Moore machine.
  - $\lambda' : Q \times \Sigma \rightarrow O$ : Output function mapping **(state, input symbol) to output**.

## • 3. Difference Between Moore and Mealy Machine:

| Difference                | Mealy Machine                                                                                   | Moore Machine                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Output Dependency</b>  | Output depends on both the current state and the current input.                                 | Output depends solely on the current state, regardless of the input.              |
| <b>Output Association</b> | Output is associated with the <b>transition between states</b> (on the arrow).                  | Output is associated with the <b>states themselves</b> (inside the state circle). |
| <b>Generality</b>         | More general and can represent a wider range of behaviors.                                      | Less general than Mealy machines.                                                 |
| <b>Implementation</b>     | Can be more complex to design and implement in some cases.                                      | Generally simpler to design and implement.                                        |
| <b>Output Changes</b>     | Output can change when either the state changes or the input changes (while in the same state). | Output can only change when the state changes.                                    |

## 1.6 Inter-Conversion: Moore and Mealy Machines

### • Moore to Mealy Conversion:

- **Rule:** Put the output of the **DESTINATION state** directly onto the transition arrow in the Mealy machine.
- **Example:** If in Moore machine  $A \xrightarrow{b} B$  and state  $B$  has output 1 ( $\lambda(B) = 1$ ), then in Mealy machine the transition becomes  $A \xrightarrow{b, 1} B$ .
- **State Count:** Number of states remains the same ( $|Q_{\text{Moore}}| = |Q_{\text{Mealy}}|$ ).

### • Mealy to Moore Conversion:

- **Rule (State Splitting):**
  - Inspect the incident (incoming) edges for each state:
  - If a state has incoming edges with **only one output** (e.g., all incoming edges have output 0), do **not** split the state.
  - If a state has incoming edges with **different outputs** (e.g., outputs 0 and 1), **split the state** into two states:  $q_{20}$  (with output 0) and  $q_{21}$  (with output 1).
- **State Count:** Number of states **increases** in Mealy  $\rightarrow$  Moore conversion.

## 1.7 Conversion from NFA with $\varepsilon$ to DFA

### STEPS:

#### Step-by-Step Procedure:

- **Step 1:** Take the  $\varepsilon$ -closure of the starting state of NFA as the starting state of DFA ( $A = \varepsilon - \text{CLOSURE}(q_0)$ ).
- **Step 2:** Find the states for each input symbol that can be traversed from the present state: compute the union of transitions and their  $\varepsilon$ -closures:

$$\delta'(A, a) = \varepsilon - \text{CLOSURE}(\delta(A, a))$$

- **Step 3:** If a new composite state is found, add it as a current state in DFA and repeat Step 2.
- **Step 4:** Repeat Step 2 and Step 3 until there are no new states generated in the transition table.
- **Step 5:** Mark all states of DFA that contain any final state of the NFA as **final states**.

## UNIT 2: REGULAR EXPRESSIONS AND LANGUAGES

### 2.1 Regular Expressions Fundamentals & Regular Languages

#### • Definition:

- The language accepted by finite automata can be easily described by simple algebraic expressions called **Regular Expressions (RE)**. It is the most effective way to represent any language.
- The languages accepted by some regular expression are referred to as **Regular Languages**.
- A regular expression can also be described as a sequence of patterns that defines a string.
- Regular expressions are used to match character combinations in strings (used in string searching algorithms).

#### • Basic Notations:

- In a regular expression,  $x^*$  (Kleene star) means **zero or more occurrences** of  $x$ . It generates:

$$x^* = \{\varepsilon, x, xx, xxx, xxxx, \dots\}$$

- In a regular expression,  $x^+$  (Positive closure) means **one or more occurrences** of  $x$ . It generates:

$$x^+ = \{x, xx, xxx, xxxx, \dots\}$$

### 2.2 Operations on Regular Languages and Expressions

1. **Language Denoted by RE:** If  $X$  is an RE over a language,  $L(X)$  represents the language denoted by  $X$ .
2. **Union ( $X + Y$  or  $X \cup Y$ ):**
  - **Intuition:** "I want Breakfast OR Lunch!"  $\Rightarrow$  Give anything from Breakfast menu OR Lunch menu.
  - **Meaning:**  $L(X + Y) = L(X) \cup L(Y)$  (either  $X$ , or  $Y$ , or both).
  - **Formal Set Definition:**  $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$ .
3. **Concatenation ( $XY$  or  $X \cdot Y$ ):**
  - **Intuition:** "First Tea  $\Rightarrow$  then Biscuit."
  - **Meaning:**  $L(XY) = L(X)L(Y)$  (first  $X$ , followed by  $Y$ ).
  - **Formal Set Definition:**  $A \cdot B = \{x \cdot y \mid x \in A \text{ and } y \in B\}$ .
4. **Kleene Star ( $X^*$ ):**
  - **Intuition:** "Give me Tea zero or more times."
  - **Meaning:**  $X^*$  means  $X$  can come 0 times, 1 time, 2 times, 3 times, ...
  - **Formal Set Definition:**  $A^* = \{x_1 x_2 \dots x_k \mid k \geq 0 \text{ and } x_i \in A\}$ .

### 2.3 Arden's Theorem

#### • Theorem Statement:

**Arden's Theorem:**

If  $P$  and  $Q$  are two regular expressions over  $\Sigma$ , and if  $P$  does not contain  $\epsilon$ , then the following equation in  $R$  given by:

$$R = Q + RP$$

has a unique solution, i.e.,

$$R = QP^*$$

**Meaning:** Whenever we obtain any state equation in the form  $R = Q + RP$ , we can directly replace it with  $R = QP^*$ .

- Mathematical Explanation / Step-by-Step Derivation:**

$$R = Q + RP$$

Substituting  $R = Q + RP$  into itself:

$$R = Q + (Q + RP)P = Q + QP + RP^2$$

Substituting again:

$$R = Q + QP + (Q + RP)P^2 = Q + QP + QP^2 + RP^3$$

In general, factoring out  $Q$ :

$$R = Q(P^0 + P^1 + P^2 + P^3 + \dots) = QP^*$$

- Rules for Writing Equations from Finite Automata:**

- For initial state (e.g.  $A$  or  $q_1$ ), add  $+\epsilon$  to the equation (since  $\epsilon$ -move represents starting at initial state).
- Dead states / trap states do not need equations because they cannot reach accepting states.
- Final regular expression is the union/equation for the final accepting state(s).

## 2.4 Conversion of RE to FA using Direct Method

The **Direct Method** builds a small Finite Automaton directly matching the structure and pattern of the regular expression:

| Operation                  | RE Pattern       | FA Construction Structure                                                                |
|----------------------------|------------------|------------------------------------------------------------------------------------------|
| <b>One Symbol</b>          | $RE = a$         | $q_0 \xrightarrow{a} q_1$ (Single transition on symbol $a$ to final state).              |
| <b>Sequence</b>            | $RE = ab$        | $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$ (Sequential transitions on $a$ then $b$ ). |
| <b>Choice (Union)</b>      | $RE = a + b$     | Two parallel branching paths from $q_0$ on $a$ and $b$ to final state $q_1$ .            |
| <b>Zero or more a's</b>    | $RE = a^*$       | Start state $q_0$ is final with self-loop on $a$ : $q_0 \xrightarrow{a} q_0$ .           |
| <b>Zero or more a, b's</b> | $RE = (a + b)^*$ | Start state $q_0$ is final with self-loops on $a$ and $b$ .                              |

## 2.5 Pumping Lemma for Regular Languages

- Purpose of Pumping Lemma:**

- Pumping Lemma is used to **prove that a language is NOT REGULAR**.
- It **CANNOT** be used to prove that a language is Regular.

- Statement of Pumping Lemma:** If  $A$  is a Regular Language, then  $A$  has a **Pumping Length** ' $p$ ' such that any string ' $s$ ' where  $|s| \geq p$  may be divided into 3 parts  $s = xyz$  such that the following 3 conditions must be true:

- $xy^iz \in A$  for every  $i \geq 0$
- $|y| > 0$
- $|xy| \leq p$

• **Proof by Contradiction Method (Step-by-Step Algorithm):**

1. **Assume** that language  $A$  is Regular.
2. It must have a pumping length (say  $p$ ).
3. All strings longer than  $p$  can be pumped ( $|s| \geq p$ ).
4. Now find/choose a clever test string ' $s$ ' in  $A$  such that  $|s| \geq p$ .
5. Divide  $s$  into  $xyz$ .
6. Show that  $xy^iz \notin A$  for some  $i$ .
7. Consider all ways that  $s$  can be divided into  $xyz$  satisfying  $|y| > 0$  and  $|xy| \leq p$ .
8. Show that **none of these can satisfy all 3 pumping conditions at the same time.**
9. String  $s$  cannot be pumped  $\Rightarrow$  **CONTRADICTION.**
10. Therefore, language  $A$  is **NOT Regular.**

## 2.6 The 12 Standard Identities of Regular Expressions

| No. | Identity                                                      | No. | Identity                                        |
|-----|---------------------------------------------------------------|-----|-------------------------------------------------|
| 1.  | $\emptyset + R = R$                                           | 7.  | $RR^* = R^*R$                                   |
| 2.  | $\emptyset R = R\emptyset = \emptyset$                        | 8.  | $(R^*)^* = R^*$                                 |
| 3.  | $\varepsilon R = R\varepsilon = R$                            | 9.  | $\varepsilon + RR^* = \varepsilon + R^*R = R^*$ |
| 4.  | $\varepsilon^* = \varepsilon$ and $\emptyset^* = \varepsilon$ | 10. | $(PQ)^*P = P(QP)^*$                             |
| 5.  | $R + R = R$                                                   | 11. | $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$        |
| 6.  | $R^*R^* = R^*$                                                | 12. | $(P + Q)R = PR + QR$ and $R(P + Q) = RP + RQ$   |